

ChezaStemEd Student Helper AI Assistant

A Serverless AI-Powered Tutoring Chatbot on AWS Cloud

Project Report

Prepared by: Dominic Cheruiyot

Date: August 2026

Abstract

ChezaStemEd Student Helper AI Assistant is a serverless, cloud-native chatbot designed to provide on-demand Science, Technology, Engineering, and Mathematics (STEM) tutoring support to students through familiar messaging platforms, namely Discord and WhatsApp. Built entirely on Amazon Web Services (AWS), the system uses Amazon API Gateway to receive student queries, AWS Lambda to orchestrate business logic, and Amazon Bedrock or the OpenAI API to generate accurate, context-aware responses. Class notes and study materials are stored in Amazon S3 as a knowledge base, while Amazon DynamoDB retains conversation history and user session data for continuity. Amazon EC2 hosts a lightweight administrative dashboard, and Amazon CloudWatch provides end-to-end monitoring, logging, and alerting. Continuous integration and deployment is achieved through GitHub Actions and AWS CodePipeline. The result is a scalable, cost-efficient, and secure assistant that reduces the burden on human tutors while giving students instant, always-available academic support outside the classroom.

1. Problem Statement

Across many secondary schools and colleges, STEM subjects remain among the most challenging areas of the curriculum, yet access to timely academic support is limited. In a typical institution serving over 200 students, teacher-to-student ratios make one-on-one tutoring impractical, and students who struggle with homework, revision, or exam preparation outside school hours have few reliable channels for help. Existing options such as physical tuition, printed revision materials, or generic search engines are either costly, slow, or fail to reference the specific class notes and syllabus content the student is being examined on. Consequently, many students fall behind, lose confidence in STEM subjects, and underperform relative to their potential. There is a clear need for an always-available, low-cost support system that can answer student questions instantly, ground its answers in the school's own class notes, and integrate with communication channels students already use daily, such as WhatsApp and Discord, rather than requiring installation of a new dedicated application.

2. Objectives

General Objective

To design and deploy a serverless AI-powered student helper assistant on AWS that delivers instant, accurate STEM tutoring support to students of a target school through WhatsApp and Discord.

Specific Objectives

- To build a scalable, serverless backend using AWS Lambda and API Gateway that can handle concurrent queries from 200+ students without performance degradation.
- To integrate a large language model (Amazon Bedrock / OpenAI API) capable of generating accurate, syllabus-aligned STEM responses.
- To create a searchable knowledge base in Amazon S3 from existing class notes so that AI responses stay grounded in the school's own curriculum.
- To persist conversation history and session data in Amazon DynamoDB for continuity and future analytics.
- To provide an administrative dashboard on Amazon EC2 for monitoring usage and managing content.
- To implement robust security controls, monitoring, and automated CI/CD deployment for reliable operation.
- To evaluate system cost, performance, and security through structured testing before wider rollout.

3. System Architecture

The diagram below illustrates the end-to-end cloud architecture of ChezaStemEd. Students interact with the system through Discord or WhatsApp. Requests are routed through Amazon API Gateway to an AWS Lambda function that runs the core AI orchestration logic. Lambda retrieves relevant class notes from Amazon S3 and calls Amazon Bedrock or the OpenAI API to generate a response. Each exchange is logged to Amazon DynamoDB, and Amazon CloudWatch monitors all events, logs, and metrics across the stack. Amazon EC2 hosts the admin dashboard and background utilities, while GitHub Actions and AWS CodePipeline automate build and deployment of the Lambda function and EC2 host.

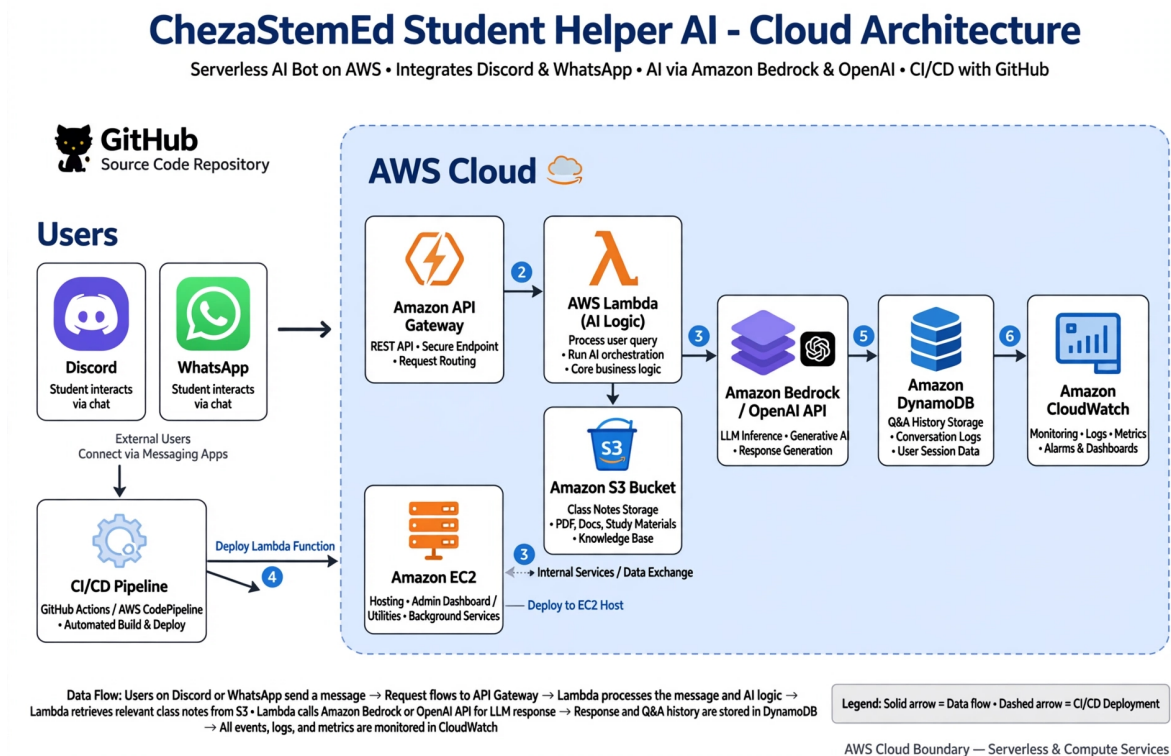


Figure 1: ChezaStemEd Student Helper AI — Cloud Architecture Diagram

Data Flow Summary

- Student sends a message on Discord or WhatsApp.
- The message is routed to Amazon API Gateway as a secure REST request.
- AWS Lambda processes the query and runs the core AI orchestration logic.
- Lambda retrieves relevant class notes and study materials from the Amazon S3 knowledge base.
- Lambda calls Amazon Bedrock or the OpenAI API for LLM inference and response generation.
- The response and Q&A history are stored in Amazon DynamoDB.
- All events, logs, and metrics are captured in Amazon CloudWatch for monitoring.
- GitHub Actions / AWS CodePipeline deploy Lambda updates and push changes to the EC2-hosted admin dashboard.

4. Tools and Technologies Used

Component	Service / Tool	Purpose
Messaging Interface	Discord, WhatsApp	Channels through which students interact with the assistant
API Layer	Amazon API Gateway	Secure REST endpoint and request routing
Compute / Logic	AWS Lambda	Serverless AI orchestration and core business logic
Knowledge Base	Amazon S3	Storage for class notes, PDFs, and study materials
AI / LLM Inference	Amazon Bedrock / OpenAI API	Generative AI response generation
Data Storage	Amazon DynamoDB	Q&A history, conversation logs, session data
Hosting	Amazon EC2	Admin dashboard and background utilities
Monitoring	Amazon CloudWatch	Logs, metrics, alarms, and dashboards
CI/CD	GitHub Actions / AWS CodePipeline	Automated build and deployment
Source Control	GitHub	Version control and source code repository

5. Cost Estimate

The estimate below assumes a pilot deployment supporting approximately 200-300 active students, each sending an average of 5-10 queries per day during school term, on AWS's pay-as-you-go pricing (us-east-1 region, 2026 rates). Actual costs will vary with usage patterns, LLM token consumption, and data volume.

AWS Service	Estimated Monthly Usage	Est. Monthly Cost (USD)
Amazon API Gateway	~60,000 requests/month	\$0.20 - \$1
AWS Lambda	~60,000 invocations, avg 2s runtime	\$3 - \$8
Amazon S3	~2-5 GB storage, low request volume	\$0.50 - \$2
Amazon Bedrock / OpenAI API	~60,000 queries, small-to-mid model	\$30 - \$80*
Amazon DynamoDB	On-demand, ~60,000 read/write ops	\$2 - \$6
Amazon EC2 (t3.micro)	1 instance, 24/7	\$7 - \$9
Amazon CloudWatch	Standard logs, metrics, alarms	\$2 - \$5
GitHub Actions	Free tier (public/small private repo)	\$0 - \$5
Data Transfer (Out)	Low volume, text-based traffic	\$1 - \$3
Estimated Total		\$46 - \$119 / month

**LLM inference is the dominant cost driver and scales directly with query volume and model choice; costs can be reduced by using a smaller model for simple queries and reserving larger models for complex questions. Most services listed also qualify for the AWS Free Tier during the first 12 months, which would lower these figures further during a pilot phase.*

6. Security Considerations

- **Authentication & Authorization:** API Gateway endpoints are secured with API keys and IAM-based authorization; Lambda functions use least-privilege IAM roles scoped only to the S3, DynamoDB, and Bedrock resources they require.
- **Data Encryption:** Data at rest in S3 and DynamoDB is encrypted using AWS-managed KMS keys; data in transit is protected via HTTPS/TLS across API Gateway, Lambda, and third-party integrations.
- **Input Validation:** All incoming messages from Discord and WhatsApp webhooks are validated and sanitized before being passed to Lambda to prevent injection attacks and malformed payloads.
- **Secrets Management:** API keys and credentials for OpenAI, Discord, and WhatsApp are stored in AWS Secrets Manager or Systems Manager Parameter Store rather than in code.
- **Network Security:** The EC2 admin dashboard sits behind a security group restricting inbound access to specific IP ranges, with a bastion or VPN pattern considered for administrative access.
- **Monitoring & Auditing:** Amazon CloudWatch captures logs and metrics for all Lambda invocations and API Gateway requests; AWS CloudTrail can be enabled to audit account-level API activity.
- **Data Privacy:** Student conversation data in DynamoDB is treated as sensitive; access is restricted to authorized roles, and a data retention policy limits how long conversation logs are kept.
- **Rate Limiting:** API Gateway throttling and usage plans prevent abuse or accidental overuse from a single user or bot account.

7. Testing

The system was validated using a combination of manual and automated testing across each layer of the architecture before rollout to the student cohort.

- **Unit Testing:** Individual Lambda functions were tested in isolation using mock events to verify correct request parsing, S3 retrieval, and response formatting.
- **Integration Testing:** End-to-end message flow was tested from Discord/WhatsApp through API Gateway, Lambda, S3, and Bedrock/OpenAI to DynamoDB, confirming data consistency at each stage.
- **Load Testing:** Simulated concurrent requests representing 200+ students were sent to API Gateway to confirm Lambda auto-scaling and acceptable response latency under peak load.
- **Security Testing:** IAM role permissions were reviewed for least privilege, and endpoints were tested against common attack patterns such as injection and unauthorized access attempts.
- **User Acceptance Testing (UAT):** A small pilot group of students used the assistant for real STEM questions, providing feedback on response accuracy, relevance, and response time.
- **Monitoring Validation:** CloudWatch alarms were verified to trigger correctly on error-rate and latency thresholds during test failure scenarios.

8. Future Improvements

- Expand the knowledge base to cover additional subjects and grade levels beyond the initial STEM pilot.
- Introduce a feedback loop where students rate response quality, feeding into periodic prompt and model tuning.
- Add support for additional channels such as Telegram or SMS for students with limited data access.
- Implement a Retrieval-Augmented Generation (RAG) pipeline with a vector database (e.g., Amazon OpenSearch or Kendra) for more precise retrieval from class notes.
- Migrate the admin dashboard from EC2 to a fully serverless option (e.g., AWS Amplify or a static site on S3 + CloudFront) to further reduce cost and operational overhead.
- Add multi-language support to serve non-English-speaking student populations.
- Introduce usage analytics dashboards for teachers to identify common topics students struggle with.
- Set up automated cost anomaly detection using AWS Budgets and Cost Anomaly Detection.